PYTHON | Front — Build small useful tools

Python 3 • examples use finding records and lab evidence • shell commands are labelled • general functions before integrations

COMMAND / PATTERN	WHAT / WHEN / CHECK
01 PROJECT / RUN / DEBUG	

`python3 -m venv .venv`
`source .venv/bin/activate`
`python -m pip install requests pandas`
	SHELL: isolate packages. requests/pandas optional.

`python -m py_compile tool.py` `python -m pdb tool.py`	SHELL: syntax/debug. pdb: n, s, p expr, c, q.
02 VALUES / COLLECTIONS	

`ports = [22, 80, 443]`
`host = {"ip": "192.0.2.10", "ports": ports}`
`host["ports"][1]`
	Nested access: index 1 returns port 80.

`host.get("os", "unknown")`
`for key, value in host.items():`
`  print(key, value)`
	get default is for missing keys, not present None.

`ports.append(8080)`
`unique = sorted(set(ports))`
`first_two = ports[:2]`
	Append mutates; set deduplicates; slice end excluded.

`ordered = sorted(ports)`
`ports.sort()`
`ports.reverse()`
	sorted returns a list; sort/reverse return None.

`host_bits = 32 - 26`
`addresses = 2 ** host_bits`
`assert addresses == 64`
	** means power; ^ means XOR. /26 gives 64 addresses.

03 CONDITIONS / TRANSFORMS	

`if 1 <= port <= 65535:`
`  print("valid port range")`
`else:`
`  raise ValueError("port out of range")`
	Validate range after converting input to an integer.

`web = [p for p in ports if p in (80, 443)]`
`for i, port in enumerate(ports, start=1):`
`  print(i, port)`
	Filter to a new list; enumerate adds a counter.

`name = raw_name.strip()`
`parts = line.split(maxsplit=2)`
`label = f"{host['ip']}:{ports[0]}"`
	Trim/split/format. Preserve case-sensitive values.

COMMAND / PATTERN	WHAT / WHEN / CHECK
04 FUNCTIONS / RETURN / TEST	

`def count_severity(records):`
`  counts = {}`
`  for r in records:`
`  level = r.get("severity", "unknown")`
`  n = counts.get(level, 0)`
`  counts[level] = n + 1`
`  return counts`
	Validated dicts/string severities assumed. Return data.

`f = count_severity`
`assert f([]) == {}`
`assert f([{}]) == {"unknown": 1}`
`r = {"severity": "high"}`
`assert f([r, r]) == {"high": 2}`
	Test empty, missing and repeated values. Not input validation.

`def add_finding(item, records=None):`
`  if records is None:`
`  records = []`
`  records.append(item)`
`  return records`
	None avoids sharing a mutable default across calls.

05 OBJECTS WHEN THEY HELP	

`class Report:`
`  def __init__(self, title):`
`  self.title = title`
`  self.findings = []`
`  def add(self, finding):`
`  self.findings.append(finding)`
	Report HAS findings. Classes organise state + behaviour.

`report = Report("Lab review")`
`report.add({"severity": "low"})`
`len(report.findings)`
	Each instance owns its list; class-level lists are shared.

06 FAIL CLEARLY / SEPARATE ENTRY POINT	

`try:`
`  port = int(raw_port)`
`except ValueError as exc:`
`  raise ValueError("Bad port") from exc`
	Catch expected errors; do not hide invalid records.

`def main():`
`  print(count_severity([]))`
 	
`if __name__ == "__main__":`
`  main()`
	Run CLI only as script; keep processing reusable.

Pathway: independent offensive tooling and CCNA automation foundations. Read sample data first; choose a schema; test edge cases before trusting a summary.

PYTHON | Back — Import, validate and report evidence

Python 3 • snippets are patterns, not one combined script • create sample files first • third-party dependencies are explicitly marked

COMMAND / PATTERN	WHAT / WHEN / CHECK
01 FILES / JSON / CSV	

`from pathlib import Path`
`p = Path("evidence") / "findings.json"`
`text = p.read_text(encoding="utf-8")`
	Read explicit UTF-8 path; missing file raises an error.

`import json`
`records = json.loads(text)`
`if not isinstance(records, list):`
`  raise ValueError("Expected a list")`
	Check JSON shape, then validate each record/field.

`out = Path("summary.json")`
`with out.open("x", encoding="utf-8") as f:`
`  json.dump(records, f, indent=2)`
	Mode x refuses overwrite; raw and derived stay separate.

`import csv`
`with open("findings.csv", newline="",`
`  encoding="utf-8-sig") as f:`
`  rows = list(csv.DictReader(f))`
	Parser handles quoted commas; verify actual encoding.

02 ADDRESSES / PARSING / INTEGRITY	

`from ipaddress import ip_address, ip_network`
`net = ip_network("192.0.2.64/26")`
`inside = ip_address("192.0.2.70") in net`
	Validate IP/CIDR; membership includes subnet boundaries.

`from urllib.parse import urlsplit, urlencode`
`u = urlsplit("https://lab.test/search?q=a")`
`query = urlencode({"q": "a b", "page": 1})`
	Parse URL, encode query; parsing does not authorise it.

`import hashlib`
`with open("raw.bin", "rb") as f:`
`  h = hashlib.file_digest(f, "sha256")`
`  digest = h.hexdigest()`
	Python 3.11+ streaming SHA-256; use trusted comparison.

`import re`
`match = re.search(r"user=(\S+)", line)`
`user = match.group(1) if match else None`
	Known log format; keep unmatched lines for review.

03 HTTP — EXPLICIT REQUEST / RESPONSE	

`import requests  # third-party`
`r = requests.get(`
`  "https://lab.test/api/items",`
`  params={"limit": 10}, timeout=10,`
`  allow_redirects=False)`
`print(r.status_code, r.headers)`
	Third-party. Lab URL, timeout, TLS on; inspect redirects.

`r.raise_for_status()` `data = r.json()`	Expected-success flow; preserve errors before raising.

COMMAND / PATTERN	WHAT / WHEN / CHECK
04 CLI / SUBPROCESS	

`import argparse`
`p = argparse.ArgumentParser()`
`p.add_argument("input_file", type=Path)`
`args = p.parse_args()`
	Path import on left. Explicit positional CLI input.

`import subprocess`
`result = subprocess.run(`
`  ["sha256sum", str(args.input_file)],`
`  capture_output=True, text=True,`
`  check=True, timeout=10)`
	Linux. Argument list, no shell; handle errors/timeouts.

05 PANDAS — OPTIONAL LARGER CSV WORK	

`import pandas as pd  # third-party`
`df = pd.read_csv(`
`  "findings.csv", dtype="string")`
`print(df.columns, df.shape)`
`print(df.isna().sum())`
	Inspect host/severity/port/seen columns and missing data.

`df["port"] = pd.to_numeric(`
`  df["port"], errors="coerce")`
`bad = df[df["port"].isna()]`
`valid = df[df["port"].between(1, 65535)]`
	Inspect invalid ports. Range alone accepts fractions.

`mask = df["severity"].eq("high")`
`high = df.loc[mask, ["host", "port"]]`
`counts = df.groupby(`
`  "host", dropna=False).size()`
	Select fields; count records, including missing hosts.

`keys = ["host", "port"]`
`unique = df.drop_duplicates(keys)`
`combined = pd.concat(`
`  [df, extra], ignore_index=True)`
	Dedup is a derived view; extra needs matching schema.

`df["seen"] = pd.to_datetime(`
`  df["seen"], utc=True, errors="coerce")`
`df.to_csv("derived.csv", index=False)`
	Assumes naive times are UTC; inspect NaT. Output overwrites.

06 TRUST THE TRANSFORM, NOT THE PROMPT	
`INPUT -> SCHEMA -> VALIDATE` `-> TRANSFORM -> CHECK -> OUTPUT`	Keep raw/rejected records. Never load untrusted pickle.

Pathway: CSV/JSON/API evidence handling for offensive work. No NumPy/plotting filler; stdlib first, pandas when volume/joins justify it. requests needs installation.